

It is also time to think how such a ground-breaking tool like ChatGPT will affect AI related research funding. It could lead to an even bigger pile of funding in the field of AI, which might lead to further ground-breaking tools and inventions. However, such heightened expectations could also lead to disappointment and a negative reaction from the industrial giants who fund most of this research. Well, it is not safe to make predictions in times like this about the future of AI. Maybe we should ask ChatGPT to make a prediction. However, I urge you to read the Wikipedia article titled '*AI winter*' to know more about research funding in the field of AI when exaggerated expectations are not reciprocated in reality. Also, if you still haven't used ChatGPT or DALL-E 2, I once again implore you to try them out. You will be mesmerized. Now, let us return to our humble efforts of creating simple applications based on AI and machine learning.

Keras and neural networks

In the last article, we trained our first model using the MNIST database of handwritten digits. We have also tested our model using a few scanned images of handwritten digits. In the last article itself, we had a detailed line by line explanation of the code, except for lines 16, 19, and 20. These lines of code were the core of the model and we merely stated that fact in the last article. These three lines of code defined the properties of the neural network used in our model as well as the training parameters used. In this article, we will discuss these three lines of code in detail so that we have a clearer understanding about neural networks and their training. First, consider line 16 of the code shown below, used to develop our model. Notice that though the code spans multiple lines, syntactically it is a

single statement of code.

```
model = keras.Sequential(
    [
        keras.Input(shape=input_shape),
        layers.Conv2D(32, kernel_size=(3, 3),
            activation="relu"),
        layers.MaxPooling2D(pool_size=(2, 2)),
        layers.Conv2D(64, kernel_size=(3, 3),
            activation="relu"),
        layers.MaxPooling2D(pool_size=(2, 2)),
        layers.Flatten(),
        layers.Dropout(0.5),
        layers.Dense(num_classes,
            activation="softmax"),
    ]
)
```

Let us briefly recall what lines 1 to 15 did to our model. First, we imported the necessary Python libraries like NumPy, Keras, etc. Then we fixed the number of classes, input shape, etc. Next, we loaded the MNIST data set and divided it into training data and testing data. Later, we checked whether a trained model exists in our system or not. We are doing this to avoid the time and resource consuming training process of a model, if a trained model already exists. If such a trained model does not exist, then the properties of a model for training are defined by the 16th line of code.

Now, let us go through that line of code shown above in detail. First, the type of neural network used is decided. Recall that a neural network has an input layer, a number of hidden layers and an output layer. Further, each layer has a number of nodes (neurons). We have defined a sequential model with the class *Sequential()* provided by the *Sequential* API of Keras. A sequential model is a stack of layers, where each layer has exactly one input tensor and one output tensor. This model allows us to create the neural network layer by layer. The sequential model is a relatively simple neural network model. If you want

to create a more complicated neural network then use the *Model()* class provided by the *Functional* API of Keras. The *Functional* API can handle models with shared layers and multiple inputs or outputs, if required. However, I preferred the simpler sequential model.

Now, let us try to understand the input parameters given to our sequential model. The code '*keras.Input(shape=input_shape)*' defines the shape of the input given to the input layer. In this case, the shape is (28, 28, 1) because the MNIST data set we used for training consists of a collection of 28x28 pixel grayscale images of handwritten digits. The code '*layers.Conv2D(32, kernel_size=(3, 3), activation="relu")*' defines a 2D convolution layer.

By the way, what is a convolution layer? A convolution layer applies a filter to an input that results in an activation. The integer 32 in the code specifies the parameter *filters*, which defines the number of output filters in the convolution. The second parameter *kernel_size* specifies the height and width of the 2D convolution layer. Recall that we have already discussed activation functions in general and the activation function called '*relu*' (rectified linear unit) in particular, in the 6th article in this series. Notice that a lot of other parameters can also be specified for a 2D convolution layer. But our simple example only specifies the three parameters we have discussed already. The code '*layers.MaxPooling2D(pool_size=(2, 2))*' generates a maximum pooling layer for processing 2D spatial data. This performs a down sampling of the input along its spatial dimensions by taking the maximum value over an input window for each channel of the input. The size of the input window is specified by the parameter *pool_size*, which in this case is (2, 2). Now, let us go through a simple